

RESEARCH REPORT

Networked Multi-Node Blockchain

Gunarathne G.D.H.

14/8/2026

1 Convergence After a Fork

Experiment

- ❖ To test fork resolution, node '8082' and '8080' was temporarily started without peer connection.
- ❖ Node '8080' mined Block 2 containing a transaction from Alice to Bob.
- ❖ Node '8082' independently mined its own Block 2 containing a completely different transaction from Bob to Alice.
- ❖ At this point, the network had a hard fork. Both nodes possessed a valid Block 2, but the block hashes and contents were completely different.
- ❖ Then both blocks connected as the peer. Both nodes were valid and did not crash or did anything.
- ❖ Then node '8080' mined Block 3 containing a transaction again which made the node '8080' chain longer.
- ❖ Node '8082' correctly identified that '8080' possessed a longer valid chain. The 'ResolveConflict' function executed the following reorganization :
 - It found the fork point at block 1.
 - It gathered all transactions from its own orphaned blocks and placed them back into the 'PendingPool'.
 - It adopted the longer chain from '8080'.
 - It deduplicated the 'PendingPool' against the new chain to ensure no transactions were double spent.

The network successfully converged on the longest chain, and the orphaned transaction safely returned to the mempool to be mined in Block 4.

2 Gossip Cost and Deduplication

Experiment

- ❖ In a fully connected 3-node network, a client submitted a single transaction to Node 8080.
- ❖ Without deduplication, Node 8080 would broadcast to 8081 and 8082. Node 8081 would broadcast to 8080 and 8082. Node 8082 would broadcast to 8080 and 8081. This creates an exponential explosion of network traffic $O(N^2)$.
- ❖ To prevent this, the network uses a strictly hash-based deduplication strategy. Every transaction struct incorporates a Unix nanosecond timestamp
- ❖ When Node 8080 receives a transaction, it caches the signature in a 'seenTxs' map before gossiping it. When Node 8081 gossips that same transaction back to Node A, Node A checks its 'seenTxs' map, identifies the duplicate signature in $O(1)$ time, and drops the HTTP request.
- ❖ This successfully bounds the gossip cost to exactly $N-1$ time messages per node, preventing infinite network loops while guaranteeing full propagation.

3 Design Write-up

Wire Format and Endpoints

The network uses standard JSON over HTTP. The core domain models ('Block' and 'Transaction') are natively tagged with JSON struct tags, allowing the standard 'encoding/json' library to serialize them directly onto the wire without the overhead of Protocol Buffers.

Endpoints are strictly RESTful:

- 'POST /transaction' and 'POST /block' - are used for network gossip and propagating new data.
- 'GET /blocks' and 'GET /mempool' - are used by peers to download the chain and pending transactions during synchronization.
- 'POST /register' - is used for auto-discovery, allowing new peers to connect and triggering automatic syncs.
- 'GET /peers' - retrieves the node's current address book.
- 'POST /sync' - manually forces the node to sync with all its known peers.
- 'GET /balances' - safely replays the blockchain to calculate and return all account balances.
- 'GET /mine' - triggers the Proof-of-Work loop to mine all pending transactions into a new block.
- 'GET /validate' - verifies the cryptographic integrity of the local blockchain against tampering.

Concurrency Safety

The 'seenTxs' map and the 'Peers' address book each have their own dedicated mutexes, preventing data races during high volume gossip.

Testing with 'go test -race ./...' confirms zero race conditions across the stack.

Ledger Rebuilding

In the ledger balances are never statically stored. During a chain reorganization, the 'calculateBalancesUnsafe()' function replays the entire blockchain from the Genesis block to the current head, mathematically ensuring that all account balances are perfectly accurate and trustless, completely eliminating the risk of state corruption during a fork.

4 Discussion Questions

Q1)

If two blocks are mined simultaneously, the network will temporarily fork. However, whichever side of the fork has more CPU power will find the next block faster and that becomes the longer chain, breaking the tie and forcing the slower half to reorganize.

A 51% attack occurs when a malicious actor controls more than half of the network's CPU power. Because they can solve hashes faster than the rest of the network combined, they can secretly mine a longer alternative chain that excludes their previous transactions, and broadcast it to force a reorganization, resulting in a "double spend".

Q2)

Finality is the guarantee that a transaction can never be reversed. Proof of Work never offers hard finality because there is always a non-zero mathematical probability that a longer valid chain exists somewhere in the world that could force a reorganization. Real networks mitigate this by waiting for "confirmations", which reduces the probability of a reversal to near-zero.

Q3)

Ed25519 signatures mathematically guarantee that a transaction was explicitly authorized by the owner of the private key, and that the transaction payload was not tampered with in transit.

However, signatures do not prevent a malicious peer from executing a Denial of Service (DoS) attack. A peer could spam the network with millions of validly signed transactions moving 0 coins back and forth, overloading the 'PendingPool' memory and network bandwidth. Our node defends against infinite loops using the 'seenTxn' cache, but it would currently still process the initial spam. A real network defends against this by requiring transaction fees.